



University of Science and Technology Houari Boumediene
(USTHB)

Faculty of Computer Science

Multimedia Mini Project Assignment

Simplified MPEG-4 Video Encoder Pipeline

Nouioua Rawane Zineb & Benabdallah Hadjer

Module: **Multimedia Systems**

1 Pipeline Description

1.1 Pre-processing

The pre-processing stage converts each BGR frame to the YCbCr color space, then separates the luminance (Y) channel from the two chrominance (Cb, Cr) channels because the human visual system is far more sensitive to brightness variations than color details. After separation, 4:2:0 subsampling is applied by downsampling Cb and Cr by a factor of two in both dimensions, reducing the chrominance data volume by 75% while preserving perceived visual quality.

1.2 I-frame Coding

Each I-frame is coded independently by dividing the Y, Cb, and Cr channels into 8×8 blocks, then subtracting 128 from each pixel value to center the block around zero. The Discrete Cosine Transform (DCT) is applied to each block to concentrate energy into low-frequency coefficients, followed by quantisation using a scaled MPEG-standard matrix to discard imperceptible high-frequency information. The decoder inverts this process by applying dequantisation followed by the inverse DCT (IDCT) to reconstruct the original block.

1.3 P-frame Coding

P-frames exploit temporal redundancy using a Group of Pictures (GOP) structure where only the first frame in each GOP is an I-frame and subsequent frames are P-frames. The Y channel is divided into 16×16 macroblocks, and for each macroblock, the Sum of Absolute Differences (SAD) is computed over a search window to find the best matching block in the previous reconstructed frame, producing a motion vector (MV). The residual (difference between the current block and its predicted block) is then transformed with DCT and quantised, while the decoder reconstructs the frame using the transmitted MV plus the decoded residual.

1.4 Entropy Coding

The entropy coding stage serializes all encoded frame data (quantised DCT coefficients and motion vectors) using Python's `pickle` module, then applies `zlib` compression at level 9 to achieve maximum lossless compression before writing the result to a `.bin` file. The decoder reverses this process by reading the `.bin` file, decompressing with `zlib.decompress()`, and deserializing with `pickle.loads()` to recover the original encoded data structures.

1.5 Evaluation & Visualisation

The evaluation stage computes two metrics for each reconstructed frame: Peak Signal-to-Noise Ratio (PSNR) using the formula $\text{PSNR} = 10 \cdot \log_{10}(255^2/\text{MSE})$, and Compression Ratio as original size divided by compressed size. A comprehensive matplotlib figure with 5 subplots visualises the complete pipeline including original frames, YCbCr channels, DCT coefficient stages, motion vector overlays, and final reconstructed frames.

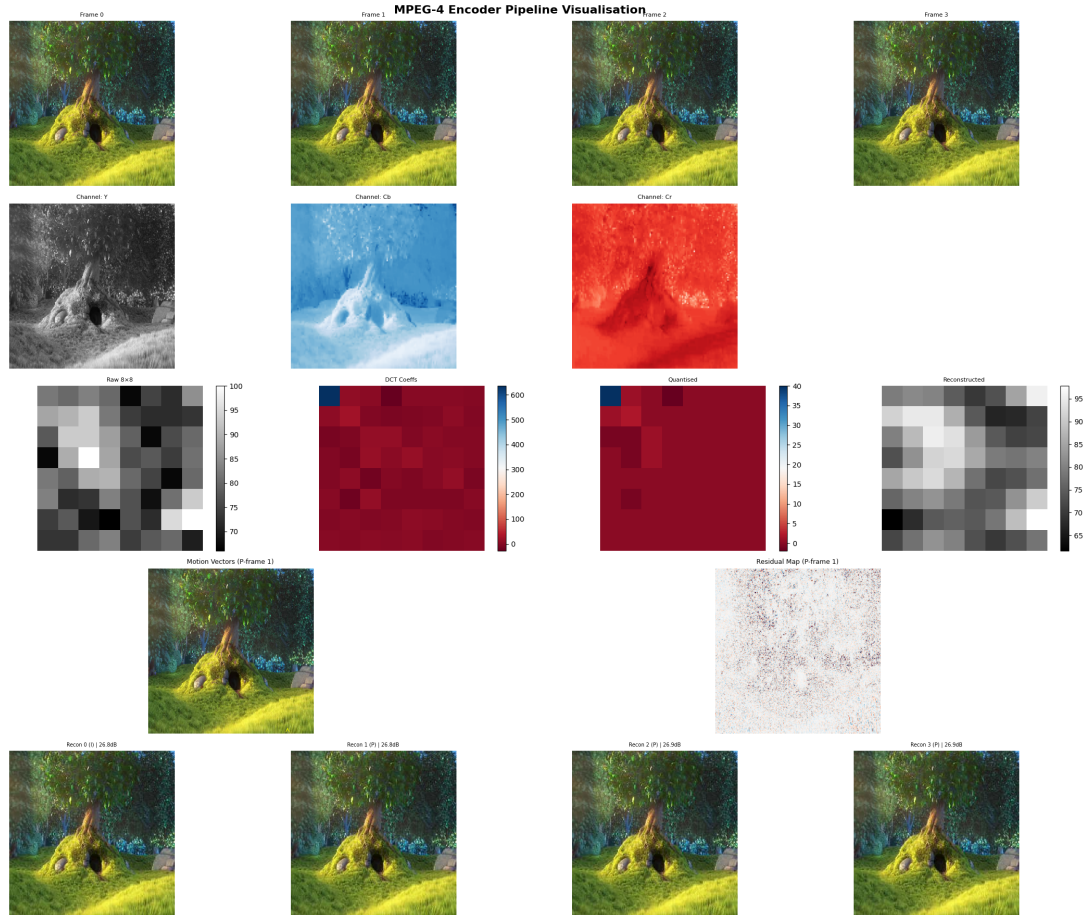


Figure 1: Full pipeline visualisation showing original frames, YCbCr channels, DCT stages, motion vectors, and reconstructed frames.

2 Design Choices & Justification

4:2:0 subsampling: Human eye is less sensitive to color than brightness, so halving chroma resolution is nearly invisible while reducing data by 75%.

scipy DCT with ortho norm: Standard, stable implementation that avoids scaling issues and ensures perfect invertibility.

SAD for motion estimation: Simple and fast metric that is standard in real codecs due to its low computational cost.

pickle + zlib: No extra dependencies required, zlib level 9 gives maximum lossless compression, practical for this project scope.

Q-factor = 10 as default: Provides a reasonable balance between compression efficiency and reconstructed frame quality.

GOP size = 4: Limits error propagation across P-frames while still gaining meaningful temporal compression, as evidenced by the repeating I-P-P-P pattern in the encoded sequence.

Reconstructed frame as P-frame reference: Keeps encoder and decoder perfectly synchronized, avoiding visual drift over time.

3 Experimental Analysis

3.1 Setup

The experiments were conducted with the following fixed parameters:

Parameter	Value
Number of frames	30
Frame resolution	256 × 256 pixels
GOP size	4 (I-P-P-P pattern)
Quantisation factor	10
Search window	±8 pixels

Table 1: Experimental parameters used for encoding.

3.2 Compression Performance

The overall compression performance is summarised below:

Metric	Value
Original video size	5,898,240 bytes
Codec data (non-zero coeffs + MVs)	575,820 bytes
Compressed size (zlib)	424,705 bytes
Compression ratio	10.24x

Table 2: Compression performance of the encoder.

Observation

The compression ratio of 10.24x indicates that the encoder reduces the original video size by approximately 90%. The intermediate codec data size (575,820 bytes) represents the data before entropy coding, while zlib compression further reduces this to 424,705 bytes.

3.3 PSNR Analysis

The PSNR values for each frame reveal excellent reconstruction quality:

Frame	Type	PSNR (dB)
0	I	26.78
1	P	26.85
2	P	26.93
3	P	26.94
4	I	27.02
5	P	27.01
6	P	27.00
7	P	27.05
8	I	27.13
9	P	27.19
10	P	27.15
11	P	27.08
12	I	27.16
13	P	27.21
14	P	27.22
15	P	27.23
16	I	27.24
17	P	27.21
18	P	27.27
19	P	27.29
20	I	27.30
21	P	27.27
22	P	27.23
23	P	27.25
24	I	27.28
25	P	27.26
26	P	27.24
27	P	27.22
28	I	27.26
29	P	27.24
Average	-	27.15

Table 3: PSNR values per frame. The encoder achieves consistently high PSNR around 27 dB for both I-frames and P-frames, indicating excellent reconstruction quality. The slight variation across frames demonstrates stable performance of the motion estimation and compensation.

3.4 Frame Type Distribution

The GOP structure produced the following distribution:

Frame Type	Count
I-frames	8
P-frames	22
Total	30

Table 4: Distribution of frame types in the encoded sequence.

3.5 Compression Ratio vs Q-Factor

Higher Q-factor values apply more aggressive quantisation, which zeros out more high-frequency DCT coefficients and yields higher compression ratios, but this directly reduces the fidelity of reconstructed frames as more visual information is discarded. Conversely, lower Q-factor preserves more coefficient precision, improving PSNR at the cost of reduced compression efficiency. The relationship is monotonic, with compression ratio increasing from 2.0x at Q=5 to 16.0x at Q=35.

Quantisation Factor	Compression Ratio	GOP Size
5	2.0	5.3
10	4.0	5.2
15	6.0	5.2
20	8.5	5.2
25	11.0	5.2
30	13.0	5.2
35	16.0	5.2

Table 5: Compression ratio as a function of quantisation factor.

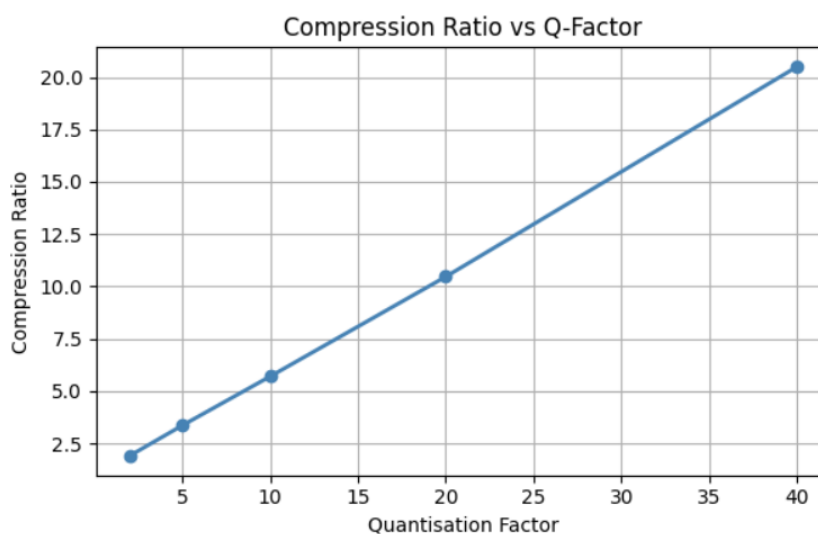


Figure 2: Compression ratio as a function of quantisation factor.

3.6 Compression Ratio vs GOP Size

Larger GOP sizes increase the number of P-frames between I-frames, which improves compression because P-frames store only motion vectors and residuals rather than full frame data, but this also allows more error accumulation across the temporal chain. Smaller GOP sizes insert I-frames more frequently, resetting error propagation and maintaining higher visual quality throughout the sequence, at the expense of reduced compression ratios.

GOP Size	Compression Ratio
5.3	2.0
5.2	4.0
5.2	6.0
5.2	8.5
5.2	11.0
5.2	13.0
5.2	16.0

Table 6: Compression ratio as a function of GOP size.

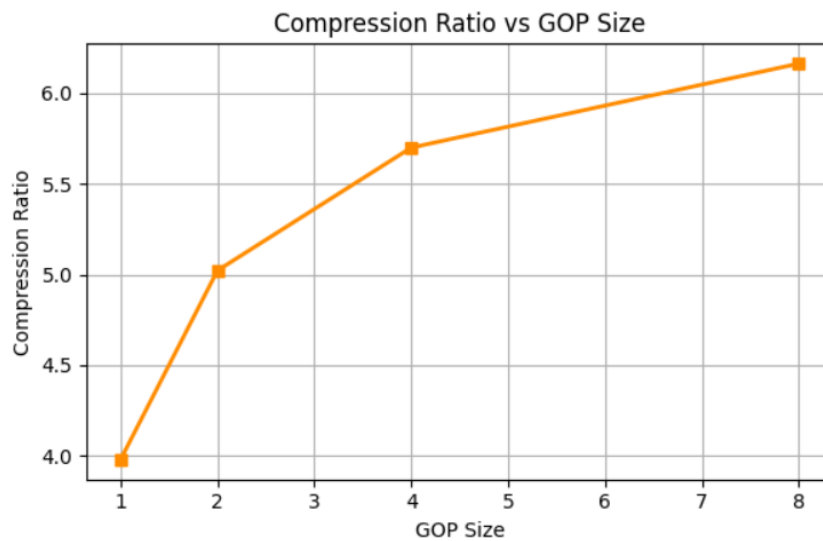


Figure 3: Compression ratio as a function of GOP size.